FP game design Waizy Giesberts

Masheed Waizy, 0384739
Kevin Giesberts, 7254970

October 2026

1 Introduction and Rules

This document describes our implementation of the game Asteroids. Asteroids is a single-player arcade shooter set in a 2D space that wraps around the screens edges. The player controls a spaceship that can rotate, boost forward, and shoot bullets to break apart moving asteroids)of many sizes) and UFOs. The game ends when the player loses all three lives by hitting an asteroid or an enemy by a UFO bullet. There is also a score tracker.

2 Design

2.1 Data structures

2.1.1 The game state

The full state of the game is stored in a GameState record. The current screen is an enumeration, so the menu, game, pause, game over, and high score screens all use one system. We also store the random number generator and high scores in the state so that all game logic can be written as pure functions on GameState.

```
data Screen = Menu | Playing | Paused | GameOver | HighScores
  deriving (Eq, Show)
data GameState = GameState
  { screen       :: Screen
  , elapsedTime  :: Float
  , ship         :: Ship
  , lives        :: Int
  , bullets      :: [Bullet]
  , asteroids    :: [Asteroid]
  , explosions   :: [Explosion]
  , rng          :: StdGen
  , score        :: Int
  , spawnTimer   :: Float
  , highScores   :: [HighScore]
  , ufos         :: [Ship]
```

```
  , ufoTimer     :: Float
  } deriving (Show, Generic)
```

The player is always human.

2.1.2 The player and the field

The ship, bullets, and asteroids exist in a flat 2D space. Positions and velocities are 2D vectors. Each entity type has its own data type so the type system keeps them separate (for example, there is always only one Ship).

```
type Vector = (Float, Float)

data TurnDir = TurnLeft | TurnRight | NoTurn
  deriving (Eq, Show, Generic)

data Ship = Ship
  { shipPos       :: Vector
  , shipVel       :: Vector
  , heading       :: Float
  , isThrusting   :: Bool
  , turnDir       :: TurnDir
  , shootCooldown :: Float
  , invulnTime    :: Float
  } deriving (Show, Generic)

data AsteroidSize = Large | Medium | Small
  deriving (Eq, Show, Generic)

data Asteroid = Asteroid
  { astPos  :: Vector
  , astVel  :: Vector
  , astSize :: AsteroidSize
  } deriving (Show, Generic)

data BulletOwner = PlayerBullet | UFOBullet
  deriving (Eq, Show, Generic)

data Bullet = Bullet
  { bulletPos      :: Vector
  , bulletVel      :: Vector
  , bulletLifeTime :: Float
  , bulletOwner    :: BulletOwner
  } deriving (Show, Generic)

data Explosion = Explosion
  { expPos        :: Vector
  , expScale      :: Float
  , expTimeLeft   :: Float
  } deriving (Show, Generic)
```

```haskell
data HighScore = HighScore
  { hsTimestamp   :: Integer
  , hsScore       :: Int
  } deriving (Show, Generic)
```

The UFO type is explained in Section 2.3. The playing field uses a continuous coordinate system that wraps around the screens edges. All number settings (screen size, turn speed, thrust, drag, bullet speed and lifetime, points per asteroid, and high score file name) are stored as named constants in a seperate Constants.hs file to avoid contamination.

2.2 Physics and Movement via Typeclasses

Instead of writing duplicate movement and collision code for every entity, we abstracted shared behavior into typeclasses (‘Collidable‘ and ‘Movable‘):

```haskell
class Collidable a where
  getPos    :: a -> Vector
  getRadius :: a -> Float

  collidesWith :: Collidable b => a -> b -> Bool
  collidesWith a b = circlesOverlap (getPos a, getRadius a) (getPos b, getRadius b)

class Collidable a => Movable a where
  getVel :: a -> Vector
  setPos :: Vector -> a -> a

  move :: Float -> a -> a
  move dt obj =
    let (x, y)   = getPos obj
        (vx, vy) = getVel obj
    in setPos (wrap (x + vx * dt, y + vy * dt)) obj
```

Screen wrap-around is written once in wrap, which is used by the generic move function from the Movable typeclass (Section 6). Any object that leaves one side of the screen appears on the opposite side with the same velocity, so the ship keeps its speed and direction. Bullets wrap the same way and they disappear when their lifetime runs out.

2.3 Hostile entities

The game has two types of enemies that behave differently: Asteroids and UFOs.

2.3.1 Asteroids

Asteroids move at a constant speed (dependent on their size). When hit by player bullets, a Large asteroid splits into two smaller medium asteroids moving in random directions. A medium asteroid splits into a small asteroid. A

small asteroid is simply destroyed. Because splitting and spawning use random numbers, we pass the random generator directly:

```
data AsteroidSize = Large | Medium | Small
  deriving (Eq, Show, Generic)

data Asteroid = Asteroid
  { astPos  :: Vector
  , astVel  :: Vector
  , astSize :: AsteroidSize
  } deriving (Show, Generic)

smaller :: AsteroidSize -> Maybe AsteroidSize
smaller Large  = Just Medium
smaller Medium = Just Small
smaller Small  = Nothing

splitAsteroid :: StdGen -> Asteroid -> ([Asteroid], StdGen)
splitAsteroid g a = case smaller (astSize a) of
  Nothing -> ([], g)
  Just sz ->
    let (ang, g1) = randomR (0, 2 * pi) g
        (spd, g2) = randomR (30, 80) g1
        v@(vx, vy) = (cos ang * spd, sin ang * spd)
        mk vel = a { astSize = sz, astVel = vel }
    in ([mk v, mk (-vx, -vy)], g2)

--Asteroid spawner controller
spawnStep :: Float -> GameState -> GameState
spawnStep dt gs
  | t > 0                                  = gs { spawnTimer = t }
  | length (asteroids gs) >= maxAsteroids = gs { spawnTimer = spawnInterval }
  | otherwise =
      let (a, g') = randomEdgeAsteroid (rng gs)
      in gs { asteroids = a : asteroids gs
            , rng        = g'
            , spawnTimer = spawnInterval }
  where t = spawnTimer gs - dt
```

2.3.2 UFOs

UFOs have basic intelligence. They move toward a target position near the player and fire a blue UFOBullet at the player in intervals. A UFO is green, and it has a simple tracking algorithm. This is their small intelligence. The neat part about UFO i that it reuses a lot of code of the player ship.

```
ufoAI :: Ship -> Ship -> Ship
ufoAI player u =
  let (ux, uy) = shipPos u
      (px, py) = shipPos player
```

```
      dx       = wrapDelta (px - ux)           -- shortest way, also across the scree
      dy       = wrapDelta (py - uy)
      hx       = sin (heading u * pi / 180)    -- the direction the UFO nose points
      hy       = cos (heading u * pi / 180)
      side     = hx * dy - hy * dx             -- < 0: player is clockwise of the nos
      ahead    = hx * dx + hy * dy             -- > 0: player is in front of the nose
      (vx, vy) = shipVel u
      turn | side < 0  = TurnLeft
           | otherwise = TurnRight
  in u { turnDir = turn, isThrusting = vx * vx + vy * vy < ufoSpeed * ufoSpeed }

  stepUFO :: Float -> Ship -> Ship -> (Ship, [Bullet])
stepUFO dt player u =
  let u' = updateShip dt (ufoAI player u)
      cd = shootCooldown u' - dt
  in case () of
       _ | cd <= 0   -> (u' { shootCooldown = ufoShootInterval }, [fireBullet UFOBull
         | otherwise -> (u' { shootCooldown = cd }, [])
```

2.3.3 Collisions

All objects have circular hitboxes, so a single generic collidesWIth function handles all checks. The collision rules are:

- Player bullet and asteroid: the asteroid splits, the player gets points, and an explosion appears.
- Player bullet and UFO: the UFO is destroyed and the player gets points.
- Ship and asteroid, UFO, or UFO bullet: the player loses a life, an explosion appears, and the ship respawns with temporary invulnerability (collisions are ignored while invulnerablity timer ¿= 0).

```
resolveCollisions :: GameState -> GameState
```

3 Implementation of the Minimum Requirements

3.1 Player

The player controls the spaceship with the arrow keys.

Player: The player controls one character using the keyboard.

Enemies: The game features enemy characters with that try to kill the lpayer. Enemies update their positions each frame in stepPure using their velocity vectors and targeting logica.

3.2 Randomness

Random numbers are used for: asteroid positions, sizes, and speeds at the start of a wave; the directions of split asteroid pieces; the screen edge where a UFO spawns; the time between UFO spawns; and the aiming error of big UFOs. Randomness stays pure because the generator is kept in the rng field of GameState, and every function that needs randomness takes a StdGen and returns an updated one.

```
randomIn      :: (Float, Float) -> StdGen -> (Float, StdGen)
randomEdgePos :: StdGen -> (Vector, StdGen)
spawnWave     :: Int -> StdGen -> ([Asteroid], StdGen)
spawnUFO      :: Int -> StdGen -> (UFO, StdGen)
```

stepPure reads rng from the state, passes it through the spawn and split functions, and saves the new generator back into the state. The initial generator is created in main using getStdGen. Because of this design, the game always behaves the same way for a given starting condition, which makes it easy to test.

3.3 Animation

Animations are done by time deltas stored in the state:

- **Explosion:** Whenever an object is destroyed. stepPure lowers expTimeLeft by dt and removes the explosion when it reaches zero. The view calculates progress as p = 1 - time left / explosion duration and draws a growing fading ring with radius expScale * p.
- **Respawn blinking:** While invulnTime is greater than 0, the ship is drawn only in alternating time intervals of elapsedTime, making it blink. This means that it blinks when it is hit to show the player it wont take damage in that state.
- **Screen transitions:** Switching between Menu, Playing, Paused, GameOver, and HighScores is done by updating the screen field.

3.4 Pause

The player can pause the game by pressing the p button. inputPure listens for this key and switches the screen between Playing and Paused. In stepPure, if the screen is Paused, all game updates are skipped, which freezes all movement and timers.

4 Implementation of the optional Requirements

We implemented one optional requirement.

4.1 Interaction with the File System and saving using JSON

Game scores are saved in a JSON file on the player's computer. The high score system uses file IO functions. The game uses the 'S' key to save a JSON file to save a gamestate. This same file can be loaded in later on by pressing the 'L' Key. The change is subtle but the scores and lives are loaded in. This way progress can be saved. When a game ends, saveHighScore saves the Unix timestamp and score to the file and updates highScores in GameState. On the high score screen, the player can view all past scores and when they were achieved.

4.2 JSON-saving

The game has three enemy types that act differently:

- **Asteroids** drift passively and split into smaller pieces when shot.
- **Big UFOs** follow the player and shoot with random aiming error.
- **Small UFOs** aim accurately and shoot faster.

From our original plan: Harder enemies show up in later waves: spawnUFO only spawns a SmallUFO from wave 6 onward, and ufoTimer gets shorter as the wave number increases. This way the difficulty keeps increasing at a steady pace.

But we realized that this was not feasible for us within time constraints. So we decided not to implement this. But it should not be too much work to implement this later on.

5 Interface and separation of pure and impure code

We use the gloss library with playIO, The game only uses impure code for three things: reading and writing the high score file, generating a random number, and closing / starting the game. Everything else should be pure code:

- `inputPure :: Event -> GameState -> GameState`: Handles state transitions and updates movement flags when keys are pressed or released.
- `stepPure :: Float -> GameState -> GameState`: Advances physics, updates bullets, runs UFO artificial intelligence, spawns new threats, and resolves collisions each frame.
- `viewPure :: GameState -> Picture`: Purely maps the current 'GameState' into a Gloss 'Picture' for rendering.

In main, we get the random generator with getStdGen, load the high scores with loadHighScores, and pass both to initial. The view and input functions simply wrap the pure functions using return . viewPure and return . inputPure e.

The score only needs to be saved once per game, not every frame. step compares the screen state before and after calling stepPure, and only saves the score when the screen changes from playing to GameOver.

6 Abstraction

Ship, Asteroid, UFO, and Bullet all move in the same 2D space, wrap around the screen, and use circular hitboxes. We use typeclasses to avoid repeating code. Each data type only defines simple helper functions, while move and collideswith are written once for all of them. This allows us to move any list of objects remove expired bullets and explosions with updateTimed, check collisions using higher-order functions like any and partition, and draw objects with map render.

7 Conclusion: What have we built?

We have build the Astroids Game. You can use SPACE or ENTER to start the game, or use H to see your highscores. Highscores are stored locally on your computer in a JSON file. In the Highscores page you see all your scores listed with the highest score at the top and the lowest score on the bottom. Behind every score is the date and time you reached that score in the format %H:%M %d/%m/%Y. From the Highscores page you can go back to the menu using either the ESCAPE key or the BACKSPACE key. Once you have started a game you can pause the game by pressing P. From the Pause menu you can go back to the menu using the B key. While playing you can move around using WASD or using the Arrow Keys. Shooting is done using the SPACE key. At any point in the game you can go back to the menu using either the ESCAPE or BACKSPACE key. In the game you have 3 lives, you lose a live when you touch an astroid or get shot by an UFO. You gain points by shooting the astroids. Once your 3 lives are over you get to see the Game Over screen. Here you can see the score you achieved in your last game and your high score. From the Game Over screen you can either restart the game by pressing SPACE or ENTER, or you can go back to the menu using ESCAPE or BACKSPACE.